

VaultX AI Internship — Week 03 Runbook

Your First Real Automation — n8n Cloud, Gmail, Google Sheets, Gemini API

Automation #1 — Inbox / Message Triage

Workflow name: Automation-01-Inbox-Triage

What it does: Watches Gmail for new incoming email. Each new email is classified by Gemini into category, priority, sentiment, and needs_human, using strict JSON output. Every email is logged to a Google Sheet. If priority is "high" or "urgent," an additional alert email is sent automatically before logging.

Trigger: Gmail Trigger, polling automatically — no manual button required.

Flow:

Gmail Trigger → Gemini (Message a Model, JSON output) → Code node (parses JSON) → If node (priority = high/urgent?) → [true: Send alert email] / [false: skip] → Merge node (preserves classification data) → Append row in Google Sheet.

Cost:

- Gemini API: gemini-3.5-flash-lite, free tier (15 requests/minute limit observed in testing)
- n8n Cloud: free tier, 1,000 executions/month
- Gmail/Sheets: no additional cost, standard Google account

How to fix it if it breaks:

- Check n8n "Executions" tab first — shows exactly which node failed and why.
- A dedicated ErrorLog Google Sheet tab automatically records every workflow failure (timestamp, failed node, error message) via a separate Error Trigger — check that first.
- Gemini, the alert email, and the Sheets write all have Retry On Fail (3 attempts) and "Continue Using Error Output" set, so a single failed API call will not crash the whole run.
- If the Sheets row shows Gmail response fields (id/threadId/labelIds) instead of classification data, the Merge node's branch order is wrong — Input 1 must be the Code node, not Send a message.
- If branching stops working (everything goes to "false"), rebuild the If condition as a single JavaScript expression rather than the UI's OR-combinator, which was found to be unreliable in testing:

```
{{ ["high","urgent"].includes($json.priority) }}
```

Automation #2 — Content / Data Pipeline

Workflow name: Automation-02-Content-Pipeline

What it does: Watches a Google Sheet (content-pipeline) for new rows containing a Title and RawContent. Gemini generates a summary, tags, and a short social-media draft from that content. Results are written back into the same row's Summary, Tags, and Draft columns.

Trigger: Google Sheets Trigger (Row Added), polling automatically once the workflow is published/active — confirmed working with zero manual intervention in testing.

Flow:

Google Sheets Trigger (row added) → Gemini (Message a Model, JSON output: summary/tags/draft) → Code node (parses JSON, re-attaches original Title/RawContent) → Update Row in Google Sheet (matched by Title).

Cost:

- Gemini API: gemini-3.5-flash-lite, free tier
- n8n Cloud: shares the same 1,000 free executions/month as Automation #1
- Google Sheets: no additional cost

How to fix it if it breaks:

- This trigger only polls automatically once the workflow is Published/Active (top-right toggle) — a workflow left in edit mode will not detect new rows on its own. This was the exact cause of an early false failure during testing.
 - A new row added before the workflow was activated will not be retroactively picked up; add a fresh test row after activation to confirm.
 - If a row's Summary/Tags/Draft come back empty, check the Code node's JSON.parse step — Gemini occasionally wraps output in extra formatting; the prompt explicitly requests "no markdown, no explanation" to reduce this risk.
 - Check n8n "Executions" tab for the specific failed node, same as Automation #1.
-

General Notes

- Both workflows use gemini-3.5-flash-lite via the Google Gemini node's built-in credential (API key), separate from the Week 1/2 Python-based google-genai SDK usage.
- Both automations were tested with genuinely new, real data (real emails sent to a real inbox; real spreadsheet rows added after activation) rather than only simulated/manual test runs.
- Structured JSON output (response as strict JSON, parsed and validated by a Code node) is the shared technique carried over from Week 2, applied here to drive real conditional branching and real data transformation rather than just demonstration scripts.